

Trabajo Práctico 1

Programación Funcional

Paradigmas de Lenguajes de Programación
1^{er} cuatrimestre 2026

Fecha de entrega: 3 de julio

Exploración de Cuevas con Drones

Una flota de microdrones científicos explora un sistema de cuevas subterráneas en busca de depósitos de cristales de alta energía. Cada dron está equipado con una batería cuya carga de energía va variando a lo largo del recorrido según los nodos que encuentra:

- Salida: el pasaje llega a la superficie. El dron emerge consumiendo tanta energía como la altura de la salida y concluye esa rama de la exploración.
- Depósito: el dron encuentra cristales y recarga tanta energía como cristales contenga el depósito, luego continúa explorando.
- Compuerta: una barrera de presión que solo se abre si la carga actual de la batería del dron satisface cierto criterio; si no lo satisface, el dron queda atrapado y la misión fracasa.
- Bifurcación: el túnel se divide en dos. El dron recorre ambas ramas: una función sobre la carga actual determina cuál se explora primero; la carga con la que termina la primera rama es la carga con la que se entra a la segunda. Cuando el dron termina de recorrer el primer camino, se teletransporta automáticamente a la bifurcación para empezar a recorrer el segundo camino.

El objetivo de los drones es relevar todos los depósitos de cristales. Se cuenta con las siguientes definiciones de tipos:

```
type Carga          = Int
type Altura         = Int
type CantidadDeCristales = Int

data Cueva = Salida      Altura
           | Depósito    CantidadDeCristales Cueva
           | Compuerta   (Carga -> Bool) Cueva
           | Bifurcación (Carga -> Dirección) Cueva Cueva

data Dirección = Izq | Der
```

Ejercicio 1

Definir y dar el tipo del esquema de recursión primitiva sobre **Cueva**. **Este es el único ejercicio en el que se permite utilizar recursión explícita.**

Ejercicio 2

Definir y dar el tipo del esquema de recursión estructural sobre **Cueva**. Se debe reutilizar el esquema definido en el ejercicio anterior.

Ejercicio 3

Definir la función `cantidadNodos :: Cueva -> Int` que devuelve la cantidad de nodos de una cueva. Ejemplos:

```
cantidadNodos (Salida 5) ~> 1
cantidadNodos (Depósito 10 (Salida 3)) ~> 2
cantidadNodos (Bifurcación (const Izq) (Salida 1) (Depósito 2 (Salida 3))) ~> 4
```

Ejercicio 4

Definir la función `profundidad :: Cueva -> Int` que devuelve la profundidad máxima de una cueva. Ejemplos:

```
profundidad (Salida 5) ~> 1
profundidad (Depósito 10 (Salida 3)) ~> 2
profundidad (Bifurcación (const Izq) (Salida 1) (Depósito 2 (Salida 3))) ~> 3
```

Ejercicio 5

Definir la función `cristalesDeDepositos :: Cueva -> [CantidadDeCristales]` que devuelve la lista con las cantidades de cristales de cada uno de los depósitos presentes en la cueva, en el orden en que se recorrerían si se eligiera siempre ir primero por la rama izquierda. Ejemplos:

```
cristalesDeDepositos (Salida 0) ~> []
cristalesDeDepositos (Depósito 5 (Salida 0)) ~> [5]
cristalesDeDepositos (Bifurcación (const Der)
                                (Depósito 3 (Salida 0))
                                (Depósito 7 (Salida 0))) ~> [3,7]
```

Ejercicio 6

Definir la función `esBienFormada :: Cueva -> Bool` que indica si la cueva está bien formada. Se dice que una Cueva está “bien formada” si ninguna compuerta aparece directamente dentro de otra compuerta (es decir, no hay dos compuertas consecutivas en ninguna rama). Esto es relevante porque el proceso de optimización del sistema asume que las compuertas ya fueron consolidadas previamente. Ejemplos:

```
esBienFormada (Salida 0) ~> True
esBienFormada (Compuerta (const True) (Salida 0)) ~> True
esBienFormada (Compuerta (const True) (Compuerta (const True) (Salida 0))) ~> False
esBienFormada (Bifurcación (const Izq)
                            (Compuerta (const True) (Salida 0))
                            (Compuerta (const True) (Compuerta (const True) (Salida 0))))
~> False
```

Justificar la elección del esquema de recursión utilizado.

Ejercicio 7

Definir la función `trasComprimirCompuertas :: Cueva -> Cueva` que devuelve una Cueva equivalente donde toda cadena de compuertas consecutivas queda reemplazada por una única compuerta cuyo predicado es la conjunción de los predicados originales. El resto de los nodos no se modifica.

Observación: el resultado de `trasComprimirCompuertas` siempre es una Cueva bien formada (no quedan Compuertas consecutivas). Se puede usar `esBienFormada` para verificarlo en los tests. Ejemplos:

```

trasComprimirCompuertas (Compuerta (const False) (Salida 7))
  ~> Compuerta (const False) (Salida 7)
trasComprimirCompuertas
  (Compuerta (\c -> c > 10)
    (Compuerta (\c -> c < 15)
      (Salida 4)))
  ~> Compuerta (\c -> c > 10 && c < 15) (Salida 4)

```

Ejercicio 8

Definir la función `cargaRestante :: Cueva -> Carga -> Maybe Carga` que describe la carga de la batería del dron al completar la exploración con una carga inicial dada, o `Nothing` si en algún momento el dron se queda sin carga o una compuerta lo atrapa. Ejemplos:

```

cargaRestante (Salida 11) 13 ~> Just 2
cargaRestante (Salida 11) 5 ~> Nothing
cargaRestante (Depósito 10 (Salida 3)) 100 ~> Just 107
cargaRestante (Compuerta (const False) (Salida 0)) 500 ~> Nothing

```

Ejercicio 9

Definir la función `pasosHastaAtrapado :: Cueva -> Carga -> Maybe Int` que describe la cantidad de nodos que el dron alcanza hasta quedar atrapado por una compuerta bloqueante o en una salida. Devuelve `Nothing` si la exploración se completa exitosamente. Ejemplos:

```

pasosHastaAtrapado (Salida 2) 50 ~> Nothing
pasosHastaAtrapado (Depósito 10 (Salida 120)) 100 ~> Just 2
pasosHastaAtrapado (Depósito 3 (Compuerta (>60) (Depósito 9 (Salida 6)))) 50
  ~> Just 2
pasosHastaAtrapado (Compuerta (<100) (Salida 75))) 50 ~> Just 2
pasosHastaAtrapado (Bifurcación (const Izq)
  (Depósito 5 (Salida 0))
  (Compuerta (const False) (Salida 0))) 50
  ~> Just 4

```

Pautas de Entrega

Se debe entregar a través del campus un único archivo llamado “tp1.zip” conteniendo el código con la implementación de las funciones pedidas. Para eso, ya se encuentra disponible la entrega “TP1 - Programación Funcional” en la solapa “TPs” (configurada de forma grupal para que sólo una persona haga la entrega en nombre del grupo). El código entregado **debe** incluir tests que permitan probar las funciones definidas. El código debe poder ser ejecutado en Haskell2010. No es necesario entregar un informe sobre el trabajo, alcanza con que el código esté **adecuadamente** comentado (son comentarios adecuados los que ayudan a entender lo que no es evidente o explican decisiones tomadas; no son adecuadas las traducciones al castellano del código). Los objetivos a evaluar son:

- Corrección.
- Declaratividad.
- Prolijidad: evitar repetir código innecesariamente y usar adecuadamente las funciones previamente definidas (tener en cuenta tanto las funciones definidas en el enunciado como las definidas por ustedes mismos).
- Uso adecuado de funciones de alto orden, curriificación y esquemas de recursión: Es necesario para los ejercicios que usen las funciones que vimos en clase y aquellas disponibles en la sección Útil del campus y aprovecharlas, por ejemplo, usar `zip`, `map`, `filter`, `take`, `takeWhile`,

`dropWhile`, `foldr`, `foldl`, listas por comprensión, etc, cuando sea necesario y no volver a implementarlas.

Salvo donde se indique lo contrario, **no se permite utilizar recursión explícita**, dado que la idea del TP es aprender a aprovechar las características enumeradas en el ítem anterior. Se permite utilizar listas por comprensión y esquemas de recursión definidos en el prelude de Haskell y los módulos `Prelude`, `List`, `Maybe`, `Data.Char`, `Data.List`, `Data.Map`, `Data.Function`, `Data.Maybe`, `Data.Ord` y `Data.Tuple`. Las sugerencias de los ejercicios pueden ayudar, pero no es obligatorio seguirlas. Pueden escribirse todas las funciones auxiliares que se requieran, pero estas no pueden usar recursión explícita (ni mutua, ni simulada con `fix`).

Tests: cada ejercicio debe contar con uno o más ejemplos que muestren que exhibe la funcionalidad solicitada. Para esto se recomienda la codificación de tests usando el paquete HUnit <https://hackage.haskell.org/package/HUnit>. El esqueleto provisto incluye algunos ejemplos de cómo utilizarlo para definir casos de test para cada ejercicio.

Importante: Se espera que la elaboración de este trabajo sea 100 % de los estudiantes del grupo que realiza la entrega. Así que, más allá de que pueden tomar información de lo visto en las clases o consultar información en la documentación de Haskell o disponible en Internet, no se podrán utilizar herramientas para generar parcial o totalmente en forma automática la resolución del TP (e.g., chat-GPT, copilot, etc). En caso de detectarse esto, el trabajo será considerado como un plagio, por lo que será gestionado de la misma forma que se resuelven las copias en los parciales u otras instancias de evaluación.

Referencias del lenguaje Haskell

Como principales referencias del lenguaje de programación Haskell, mencionaremos:

- **The Haskell 2010 Language Report:** el reporte oficial de la versión del lenguaje Haskell al 2010, disponible online en <https://www.haskell.org/onlinereport/haskell2010>.
- **Learn You a Haskell for Great Good!:** libro accesible, para todas las edades, cubriendo todos los aspectos del lenguaje, notoriamente ilustrado, disponible online en <https://learnyouahaskell.com/chapters>.
- **Real World Haskell:** libro apuntado a zanjar la brecha de aplicación de Haskell, enfocándose principalmente en la utilización de estructuras de datos funcionales en la “vida real”, disponible online en <https://book.realworldhaskell.org/read>.
- **Hoogle:** buscador que acepta tanto nombres de funciones y módulos, como signatures y tipos *parciales*, online en <https://www.haskell.org/hoogle>.